

Trabajo Práctico 2

EDyA II

Secuencias

22 de junio de 2026

Integrantes

- Milton Pacheco
- Luciano Belardo
- Juliana Altobello

Consigna

b) Para la implementación dada en a, dar la especificación de costos (trabajo y profundidad) de las funciones `filterS`, `reduceS` y `scanS`. No es necesario demostrarlo.

Especificación de costos de `filterS`

Implementación

```
filterS :: (a -> Bool) -> [a] -> [a]
filterS _ [] = []
filterS f (x:xs) =
  let (cond, xs') = f x ||| filterS f xs
  in if cond then x : xs' else xs'
```

Trabajo de `filterS` (W)

Sea $s = [s_0, s_1, \dots, s_{n-1}]$.

$$W(\text{filterS } f []) = k_1$$

$$W(\text{filterS } f (x : xs)) = W(fx) + W(\text{filterS } f xs) + k_2$$

Por lo tanto:

$$W(\text{filterS } f s) = O\left(\sum_{i=0}^{n-1} W(f s_i) + n\right)$$

Profundidad de filterS (S)

$$S(\text{filterS } f []) = k_1$$

$$S(\text{filterS } f(x : xs)) = \text{máx}(S(fx), S(\text{filterS } fxs)) + k_2$$

Por lo tanto:

$$S(\text{filterS } fs) = O\left(\text{máx}_{0 \leq i < n} S(f s_i) + n\right)$$

Especificación de costos de reduceS

Implementación

```
reduceS :: (a -> a -> a) -> a -> [a] -> a
reduceS _ e [] = e
reduceS _ _ [x] = x
reduceS f e xs = reduceS f e (contraer xs)
  where
    contraer [] = []
    contraer [x] = [x]
    contraer (x:y:rest) =
      let (a, b) = f x y ||| contraer rest
      in a : b
```

Trabajo de reduceS (W)

$$W(\text{reduceS } fe []) = k_1$$

$$W(\text{reduceS } fe[x]) = k_2$$

$$W(\text{reduceS } fexs) = W(\text{contraer } xs) + W(\text{reduceS } fe(\text{contraer } xs)) + k_3$$

Por lo tanto:

$$W(\text{reduceS } fexs) = O\left(|xs| + \sum_{(x \oplus y) \in Or(\oplus, e, xs)} W(x \oplus y)\right)$$

Profundidad de reduceS (S)

$$S(\text{reduceS } fe []) = k_1$$

$$S(\text{reduceS } fe[x]) = k_2$$

$$S(\text{reduceS } fexs) = S(\text{contraer } xs) + S(\text{reduceS } fe(\text{contraer } xs)) + k_3$$

Por lo tanto:

$$S(\text{reduceS } fexs) = O\left(|xs| + \text{máx}_{(x \oplus y) \in Or(\oplus, e, xs)} S(x \oplus y)\right)$$

Especificación de costos de scanS

Implementación

```
scanS :: (a -> a -> a) -> a -> [a] -> ([a], a)
scanS f e [] = ([], e)
scanS f e [x] = ([e], x)
scanS f e xs =
  let apareados = contraerPares xs
      (pares, totalPares) = scanS f e apareados
      paresOriginales = extraerPares xs
      prefijos = combinarS pares paresOriginales totalPares
      totalFinal = calcularTotal paresOriginales totalPares
  in (prefijos, totalFinal)
```

Trabajo de scanS (W)

$$W(\text{scanS } fe[]) = k_1$$

$$W(\text{scanS } fe[x]) = k_2$$

$$W(\text{scanS } fexs) = W(\text{contraerPares } xs) + W(\text{scanS } fe(\text{contraerPares } xs)) + W(\text{combinarS}) + W(\text{calcularTotal}) +$$

Por lo tanto:

$$W(\text{scanS } fexs) = O \left(|xs| + \sum_{(x \oplus y) \in Os(\oplus, e, xs)} W(x \oplus y) \right)$$

Profundidad de scanS (S)

$$S(\text{scanS } fe[]) = k_1$$

$$S(\text{scanS } fe[x]) = k_2$$

$$S(\text{scanS } fexs) = S(\text{contraerPares } xs) + S(\text{scanS } fe(\text{contraerPares } xs)) + S(\text{combinarS}) + S(\text{calcularTotal}) + k_3$$

Por lo tanto:

$$S(\text{scanS } fexs) = O \left(|xs| + \max_{(x \oplus y) \in Os(\oplus, e, xs)} S(x \oplus y) \right)$$

Consigna

d) Muestre que la implementación dada en (c) verifica la especificación de costos para la profundidad de la operación **reduceS**.

Veamos la profundidad de **reduceS** sobre arreglos.

La profundidad de **nthA** es constante y la profundidad de **tabulateA** es

$$O\left(\max_{i=0}^{m-1} S(f\ i)\right).$$

En nuestra implementación de **contraA**:

```
contraA :: (a -> a -> a) -> Arr a -> Arr a
contraA f arr = tabulateA faux m
where
  faux i =
    if i == mid
    then nthA arr (n - 1)
    else f (nthA arr (i*2)) (nthA arr (i*2+1))
  mid = n `div` 2
  n   = lengthA arr
  m   = if n `mod` 2 == 0 then mid else mid + 1
```

Se usa **tabulateA** con argumento **faux** y tamaño $m = \lceil n/2 \rceil$; como **nthA** cuesta $O(1)$, la profundidad de **faux** está dominada por la profundidad de **f**. Por lo tanto,

$$S(\text{contraA } f \text{ arr}) = O\left(\max_{i=0}^{\lceil n/2 \rceil - 1} S(f(\text{arr}_{2i}, \text{arr}_{2i+1}))\right).$$

La función **reduceA** se definió así:

```
reduceA :: (a -> a -> a) -> a -> Arr a -> a
reduceA f e arr =
  case lengthA arr of
    0 -> e
    1 -> f e (nthA arr 0)
    _ -> reduceA f e (contraA f arr)
```

Cada llamada a **contraA** reduce a la mitad la longitud del arreglo, de modo que el número de niveles es $O(\log |arr|)$.

En consecuencia,

$$S(\text{reduceA } f e \text{ arr}) = O\left(\log |arr| \cdot \max_{(x \oplus y) \in Or(\oplus, e, arr)} S(x \oplus y)\right),$$